Tools for software development

Basic types of tests

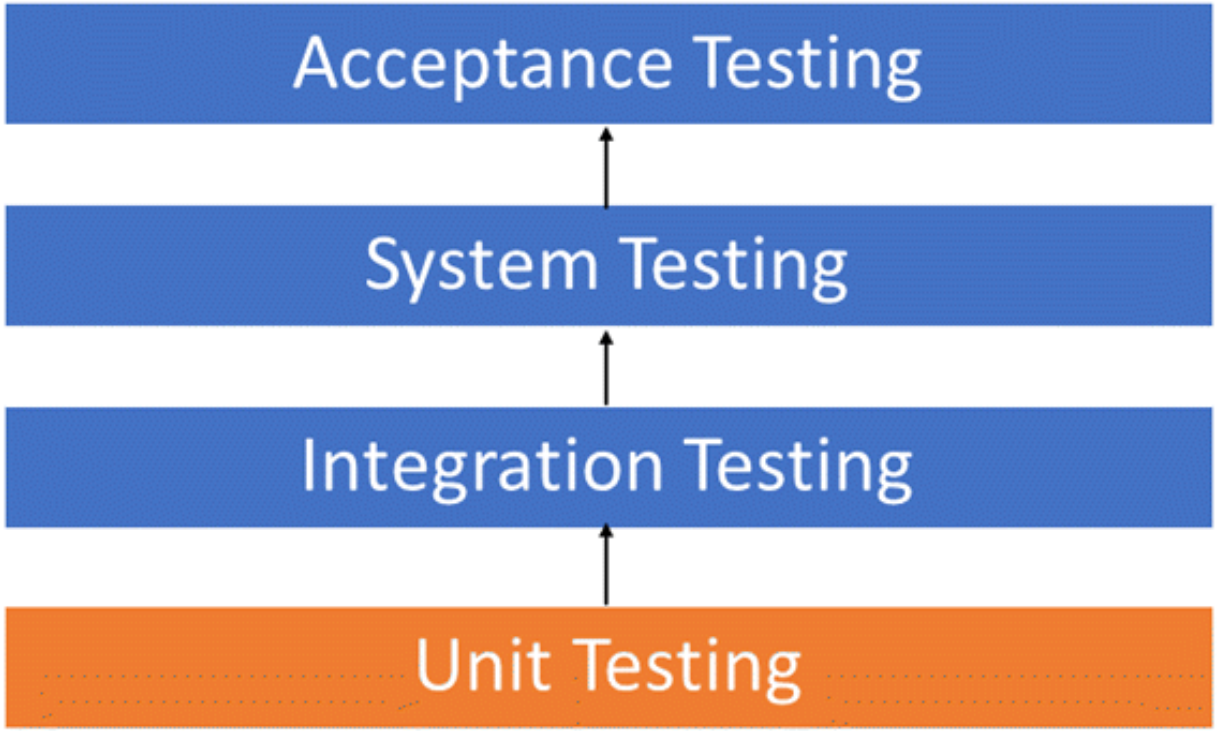
doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk

ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Test types by level

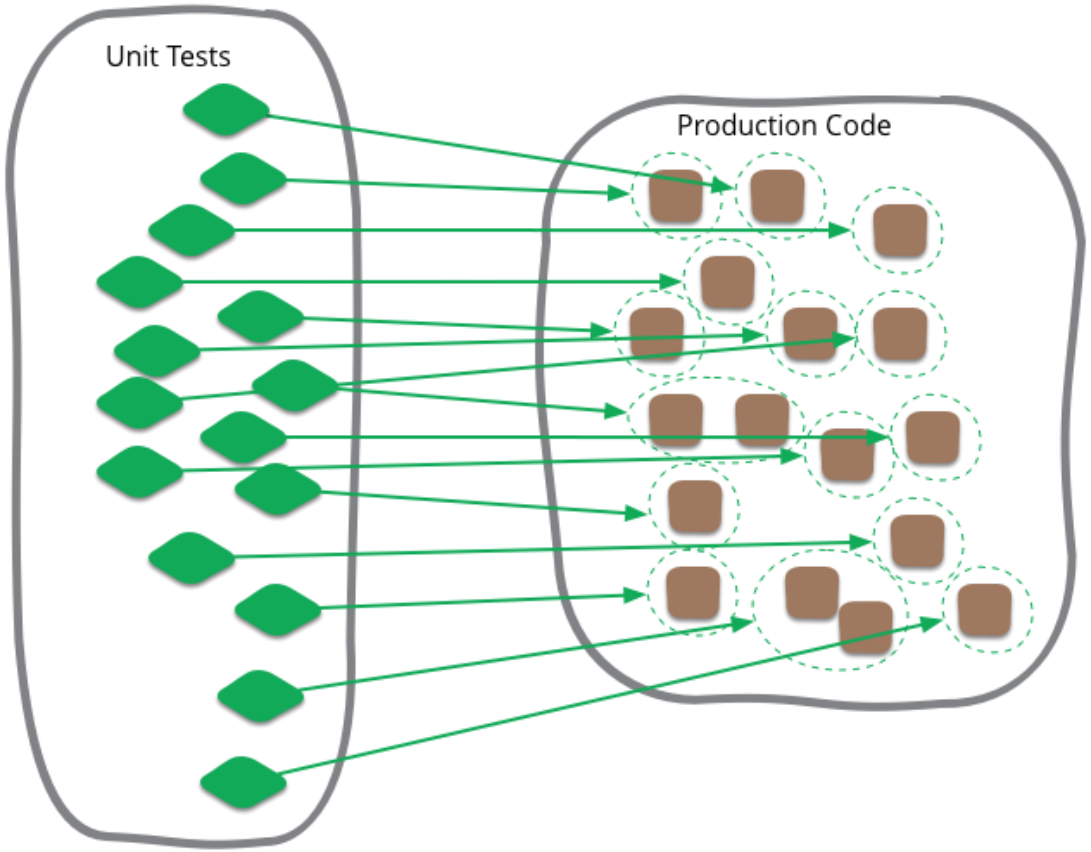
- Different types of tests at different levels of software development

- 4 levels of testing:
  - Unit testing
  - Integration testing
  - System testing
  - Acceptance testing



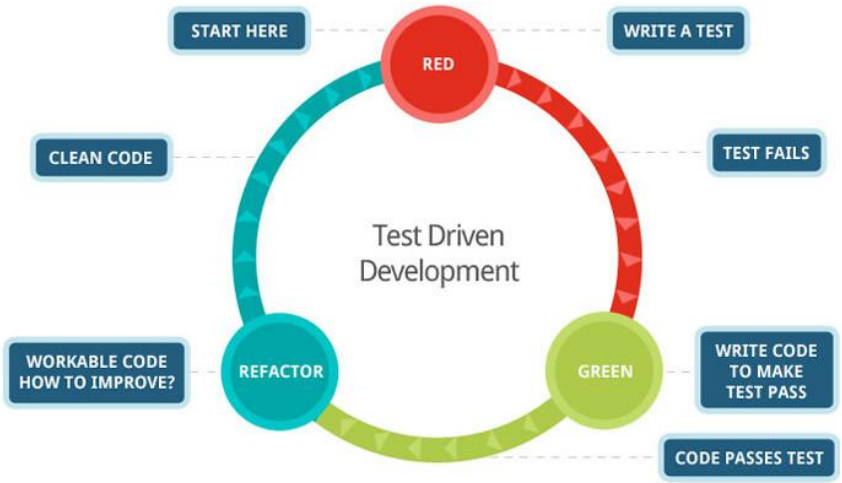
Unit testing

- The programmer himself is already testing
- Components - units
  - Smallest tested program components
- We test each component separately
  - We are looking for proof that he is behaving properly
- The need to simulate real behavior

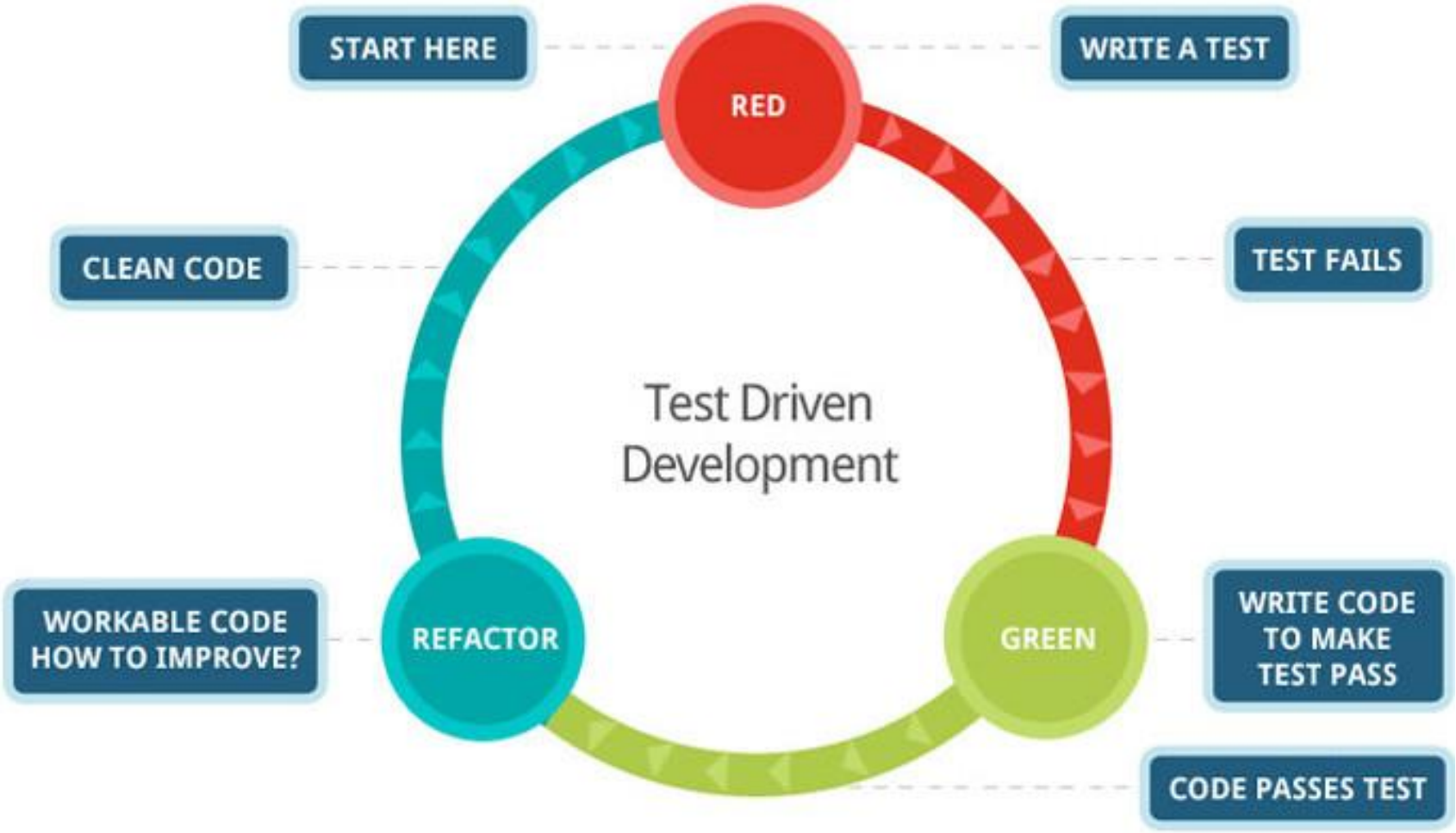


Unit testing (Component testing)

- Test-driven development (TDD)
  - They are written more like code
  - The basis of agile methodologies
- Advantageous - repeatability and automation
- They only catch specific component errors



Unit testing (Component testing)



Unit testing

Example:

- Calculating a customer discount based on age

```
public int countRebates (int ageOfCustomer )
{
     if (customerAge > 50)
          return 50;
     else if (customerAge > 40)
          return 20;
     else if (customerAge > 30)
          return 10;
     else return 0;
}
```

Unit testing

Example:

- Calculating a customer discount based on age

```
public int countRebates (int ageOfCustomer )
{
     if (customerAge > 50)
          return 50;
     else if (customerAge > 40)
          return 20;
     else if (customerAge > 30)
          return 10;
     else return 0;
}
```

```
[TEST]
public void calculationSaleOver50()
{
          int expectedAge = 50;
          int currentResult = calculateDiscounts(55);
          Assert.AreEqual(expectedAge , currentResult);
}
```

Integration testing

- Connecting components creates a system
- If the components are working
  - we integrate them into a whole
- Progress:
  - Gradual addition of components with functional unit tests
  - They must function properly
  - They must not disrupt the stability of the overall system
- Error detection
  - Created by combining modules
- When?
  - Sufficient number of modules for testing
  - Don't wait until the end

Integration testing

Example:

- calculation deposit amount on number days

- An error occurred while integrating two modules

```
class User
{
     int countDays = identifyCountDays();
     int ammountOfDeposit = identifyDeposit();
     int rate = calculatingModul.calculateRate(ammountOfDeposit, countDays);
}
```

Integration testing

Example:

- calculation interest on number days

- An error occurred while integrating two modules

```
class User
{
     int countDays = identifyCountDays();
     int ammountOfDeposit = identifyDeposit();
     int rate = calculatingModul.calculateRate(ammountOfDeposit, countDays);
}
```

```
class calculationModule
{
     public static int calculateRate(int countDays, int ammountOfDeposit)
     {
          // … formulas for calculating …
     }
}
```

Integration testing

Integration solution:

- **Big bang**
  - All modules integrated at once
  - Difficult problem detection
- ***Top - down* approach**
  - According to the hierarchical structure, we gradually add
  - Partial functioning at a certain number
- **Bottom-up approach**
  - From the lowest modules up
  - The system works until the highest one is integrated
- **Combined / sandwich**
  - Top / bottom combination
  - 3 levels:
    1. from the top application to the main modules
    2. from the bottom to the frequently used lowest modules
    3. gradual addition of other modules
  - Ability to test both main and lowest modules simultaneously

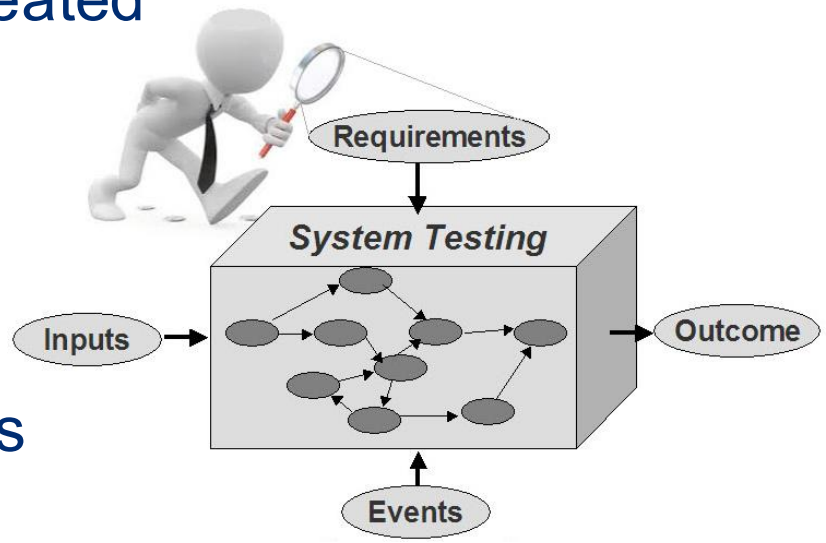
Integration testing

Possible applications:

- **Intrasystem**
  - Internal modules
- **Intersystem**
  - Combining two already tested systems into a whole
  - System Integration Tests (SIT)
    - Tests with external systems

System testing

- After integration tests, a sufficiently stable system is created
- Verification of customer-specified requirements
- Result:
  - Belief that the customer will accept the product



- Testing both functional and non-functional requirements
- Not only to find defects but also to provide assurance of functionality
- Few defects => correct method chosen?

System testing

Most frequently performed tests:

- **Functional tests**
  - Whether it meets specified and implicit requirements
  - Security tests – whether the data is sufficiently secured
- **Robustness tests**
  - System processing of incorrect inputs
  - Responding to unexpected situations
- **Usability tests**
  - Will the user be able to use the system comfortably and clearly enough?
  - Navigation, clarity, ease of use, layout of elements
- **Interoperability tests**
  - Whether it communicates properly with other systems
  - Only communication is tested, not data accuracy
- **Reliability tests**
  - How long will it work before failure?
- **Performance tests**
  - Monitoring performance during simple scenarios
  - *Stress tests – long-term stress test*
  - *Stress tests – behavior under extreme stress*
  - *Scalability tests – testing for future load increases*

Acceptance testing

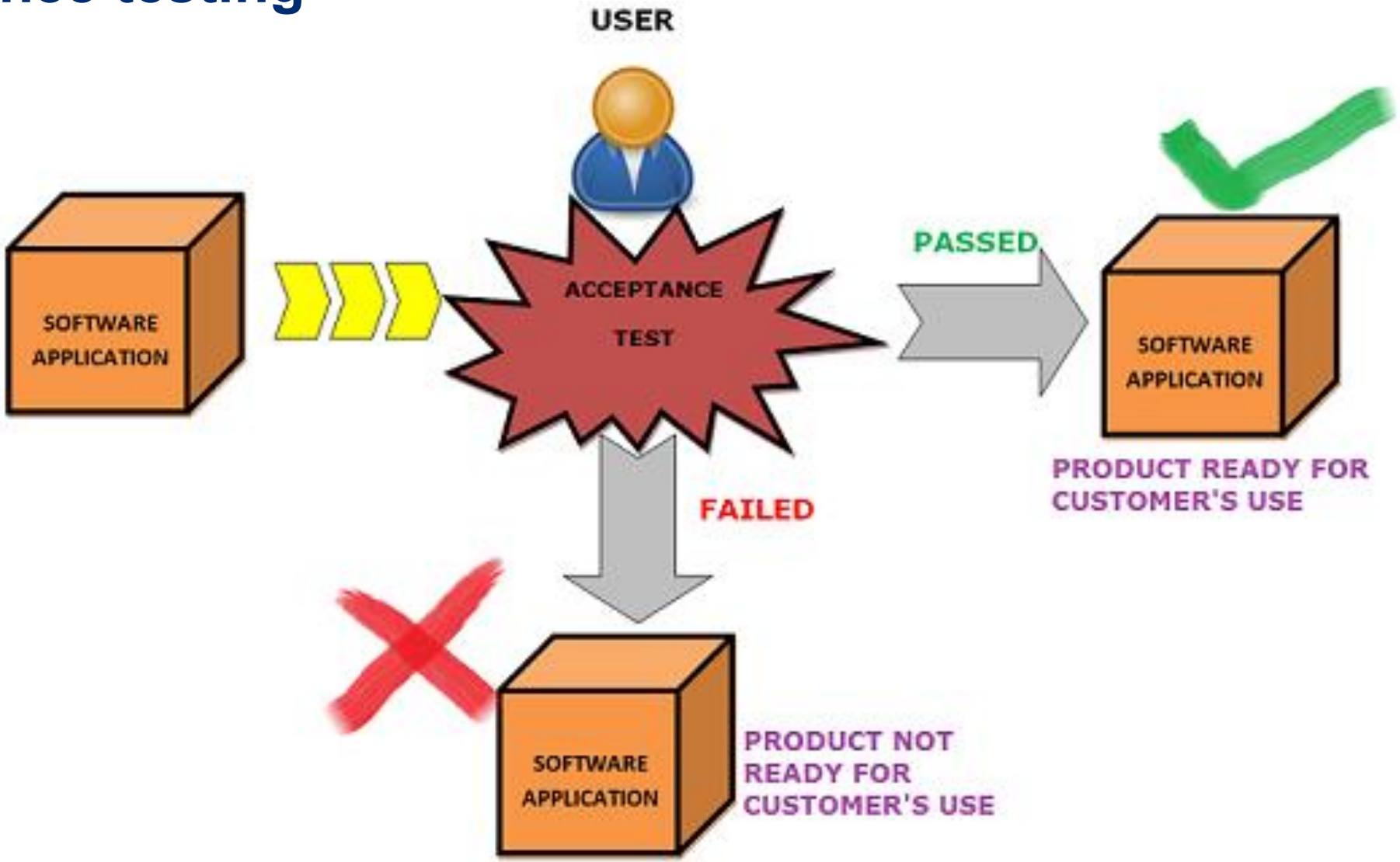
- Customer/customer user acceptance testing
  - *User Acceptance Testing (UAT)*
  - Meets acceptance criteria set by the customer
  - It is no longer just about testing, but also about verifying product acceptance – e.g. missing documentation

- Testing software, manual, templates...
- Testing in real operation
- The tester should know what the system is supposed to do and what it is used for

- Tests for compliance with laws, standards, acceptance by the chamber

- Versioning : Alpha, Beta versions

Acceptance testing



Regression and confirmation testing

- Making modifications
  - Bug fix, functionality extension, performance improvement, dead code removal
- Risk of developing a new defect

- Verifying whether the system is functional again after the change
- Using existing cases
- Easy to apply in test automation

- Creating a regression test suite
  - Coverage of all application components
- Regular execution at every build

Regression and confirmation testing

How to select test cases:

- Tests verifying basic and key functionality
- Tests covering very frequently used system components
- Tests covering the area of change and its impact
- Tests leading to the discovery of the error
- Tests already working, failed

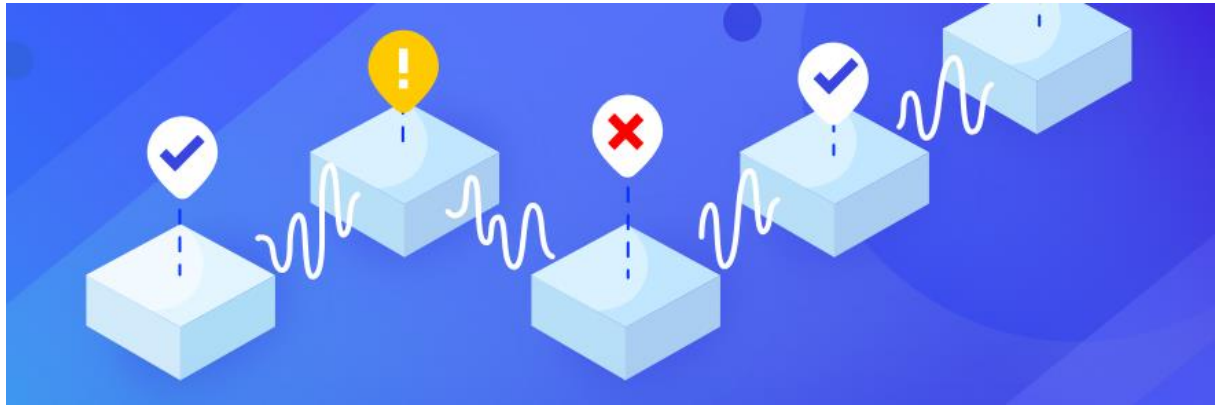
Smoke and Sanity Testing

- Verifying whether the system is stable enough and whether the main parts are working properly
- The result is a decision whether it makes sense to continue testing on the current build .
- Mostly automated tests

- **Sanity tests** - in more detail
  - Checking whether the correct data was returned, whether authorization was successful ….
- Both fall under regression testing

End-to-End testing

- Various forms
- Tracking an entity throughout its lifetime in the system
- System entry, interaction , final state



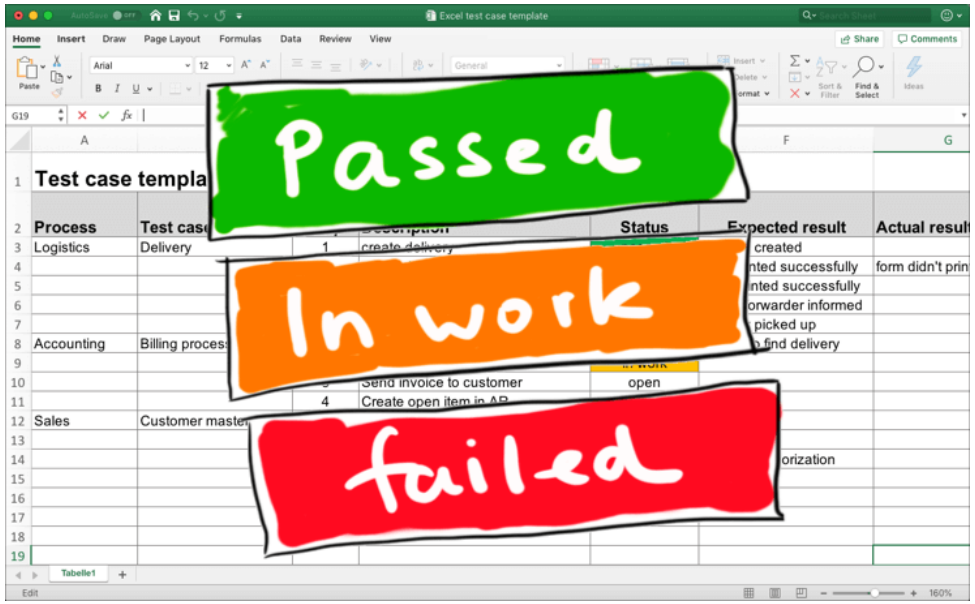
- Part of acceptance tests
  - User login, their work, logout user
- Also possible for system tests
  - Transaction and data tracking – correct display at the end
  - Confirmation that subsystems are functioning correctly throughout the entire “life cycle”

Test case -Test script

- **Test Case** (IEEE 1012:2004)
  - *A set of inputs, conditions for triggering, and expected results, developed for a specific purpose, such as executing a specific path in a program or verifying compliance with requirements.*

- We determine whether the system responds as it should under given conditions, given the specified inputs and expected outputs.

- Many ways to perform documentation
  - IEEE 829:2008



Test case -Test script

Test Case (TC):

- Unique TC identifier
- The purpose or focus of the test
- Specification of all input data to perform the test
- Specification of output values, properties and expected behavior
- Environmental needs – hardware, software ...
- Requirements for performing the test – input and output conditions, functions performed before
- Possible dependencies on other test cases that must be executed in advance

Test case -Test script

Example:

```
Case number: TC005.1
Title: Verifying the functionality of calculating the XYZ function with positive numbers
Priority: highest
Prerequisites: user is logged in, module is open
Inputs: X=10, Y=15, Z=100
Expected output: 10
Actual output: (not yet tested)
Test result: (not yet tested Passed/Failed/Not proven...)
Notes:
```

Test case -Test script

- Test Script
  - Test case group
  - Test procedure specification
    - the steps for implementing tests and their order
- Why document testing?
    - Simple regression testing to uncover new bugs
    - Greater likelihood of detecting deficiencies
    - Possibility of entrusting testers who are unfamiliar with the system
    - Possibility of checking and detecting deficiencies

Black, white and gray box testing

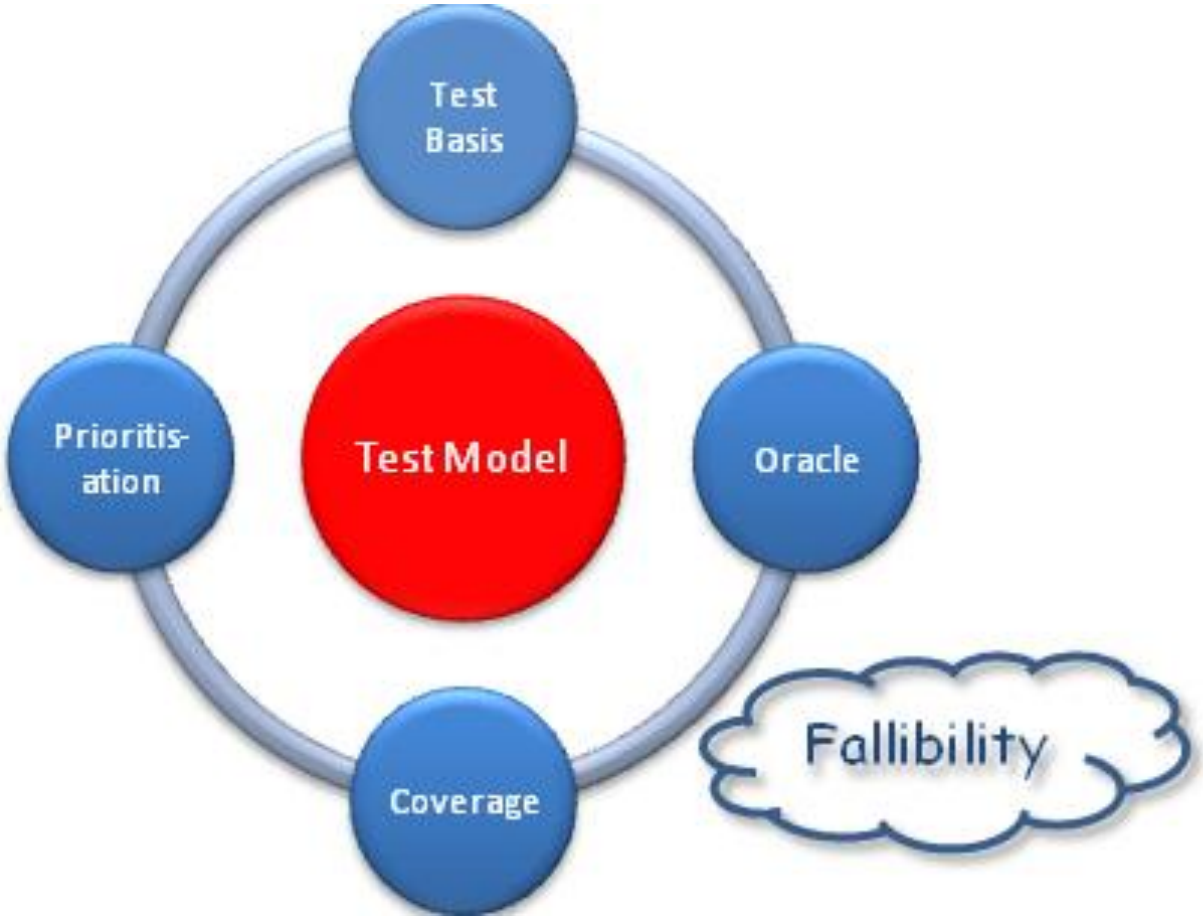
- **Black box approach**
  - Focuses on system behavior
  - Testers have no knowledge of the internal functionality of the system
  - What matters is whether the output is correct.

- **White box approach**
  - Test cases are designed based on knowledge of functionality
  - Unit tests (not always)

- **Gray box approach**
  - When designing, we have a requirements specification and development documentation
  - Development of tests focused on functionality and potential vulnerabilities
  - Minimizing the risk of situations that we would not catch with a test

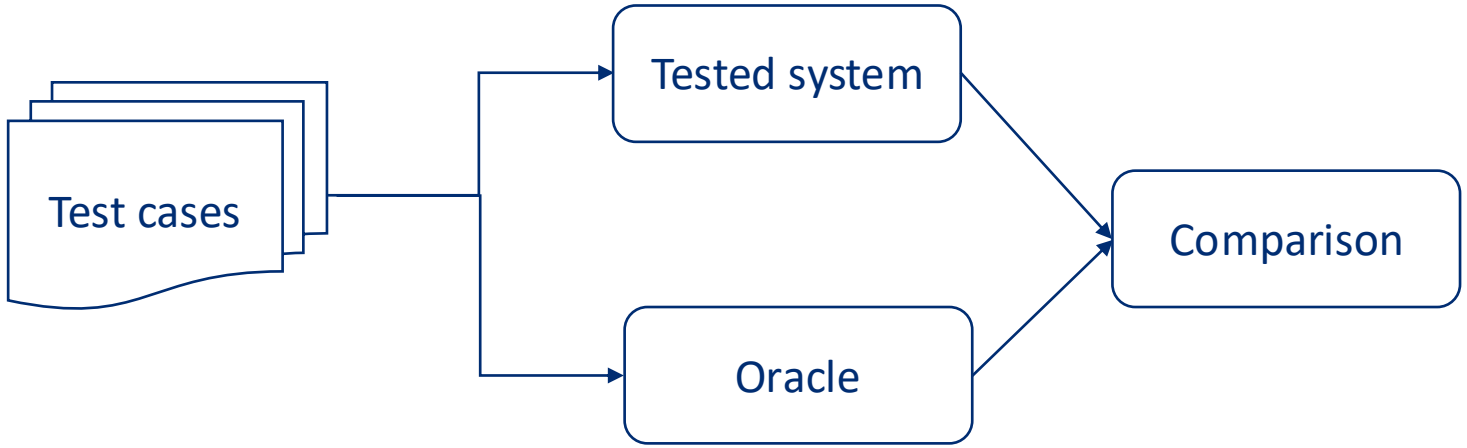
Testing oracle

- *Oracle test*
- The basic problem
  - Is the system behaving as it should?
- Design of valid test cases
  - Information about expected behavior
  - How to determine correctness
- Source of this data



Testing oracle

- A mechanism that helps determine the expected outcome of tests or their accuracy
- Specifications, accompanying documentation, older versions...



Thank you for your attention

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk